

Univerzitet u Beogradu Elektrotehnički fakultet



Digitalni VLSI sistemi
[19E044DVS]

Projekat: Ubrzanje algoritma za određivanje gradijenta
i detekciju ivica na slikama

Sadržaj

1	Softver	3
1.1	Kod	3
1.1.1	params.h	3
1.1.2	timer.h & timer.c	4
1.1.3	dma.h & dma.c	4
1.1.4	sd.h & sd.c	5
1.1.5	processing.h & processing.c	6
1.1.6	main.h	7
1.2	Dijagrami	9
1.2.1	Dijagram aktivnosti	9
1.2.2	Dijagram sekvence	10
2	Hardver	11
2.1	Komponente	11
2.1.1	axi_acc_edge_detection	11
2.1.2	acc_edge_detection	11
2.1.3	shift_reg	12
2.1.4	ram	13
2.2	Šematski prikaz	14
2.2.1	axi_acc_edge_detection	15
2.2.2	acc_edge_detection	16
2.2.3	shift_reg	17
3	Testovi	18
3.1	Izlazne slike	18
3.2	Hardversko ubrzanje procesiranja	22

1 Softver

1.1 Kod

Softversko rešenje se sastoji od 5 *source* (.c) i 5 *header* (.h) fajlova:

- ◇ `params.h`
- ◇ `timer.h` i `timer.c`
- ◇ `dma.h` i `dma.c`
- ◇ `sd.h` i `sd.c`
- ◇ `processing.h` i `processing.c`
- ◇ `main.c`

1.1.1 `params.h`

U **`params.h`** fajlu su definisani makroi za maksimalne dimenzije slike, maksimalnu dužinu imena fajla i maksimalni broj karaktera za putanju do fajla.

```
#define MAX_IMG_DIMENSION 512
#define MAX_FILENAME_LEN 255
#define MAX_PATH_LEN 260
```

Parametri se čuvaju u strukturi **Params** (**Params_t**). Polja strukture su prikazana u tabeli 1.1.

Tabela 1.1: Parametri strukture **Params_t**

Parametar	Vrednost	Opis
ImgW	4 - MAX_IMG_DIMENSION	Širina slike
ImgH	4 - MAX_IMG_DIMENSION	Visina slike
EdgeThr	0 - 255	Prag za izračunavanje ivice
Mode	EDGE (0)	Izračunavanje ivice slike
	GRAD_H (1)	Izračunavanje horizontalnog gradijenta slike
	GRAD_V (2)	Izračunavanje vertikalnog gradijenta slike
	GRAD_M (3)	Izračunavanje gradijenta magnitude slike
Border	ZERO (0)	Vrednost ivičnog elementa je 0
	CLOSEST_ELEM (1)	Vrednost ivičnog elementa je jednaka vrednosti najbližeg elementa unutar slike
Bypass	NO (0)	Procesiranje slike na osnovu datih parametara
	YES (1)	Propuštanje slike kroz sistem (zaobilaženje procesiranja)
imgName	char[MAX_FILENAME_LEN]	Naziv ulazne slike

1.1.2 timer.h & timer.c

U ovom delu je realizovan "interfejs" koji omogućava korišćenje tajmera. Pre korišćenja tajmera, potrebno je pozvati funkciju **timerConfig()** koja inicijalizuje instancu tajmera (**AxiTimerInst**) i vraća **XST_SUCCESS** nakon uspešne inicijalizacije, a u suprotnom vraća **XST_FAILURE**. Ostale funkcije predstavljaju omotač oko funkcija analognih **XTmrCtr** funkcija koje se pozivaju nad instancom **AxiTimerInst**. Kratak opis funkcija je dat u tabeli 1.2.

Tabela 1.2: Funkcije za upravljanje tajmerom

Funkcija	Opis
int timerConfig()	Inicijalizacija tajmera
void startTimer()	Pokretanje tajmera
void stopTimer()	Zaustavljanje tajmera
void resetTimer()	Resetovanje tajmera
long double getTime()	Čitanje vremena u mikrosekundama

1.1.3 dma.h & dma.c

U ovom delu je realizovan "interfejs" koji omogućava korišćenje DMA. Pre korišćenja DMA, potrebno je da se pozove funkcija **dmaConfig()** koja inicijalizuje instancu DMA kontrolera (**AxiDma**). Pre svakog prenosa podataka potrebno je resetovati DMA kontroler (**dmaReset()**). U tabeli 1.3 je dat kratak opis funkcija.

Tabela 1.3: Funkcije za konfigurisanje DMA kontrolera

Funkcija	Opis
int dmaConfig()	Inicijalizacija DMA kontrolera i konfigurisanje prekida
void dmaReset()	Resetovanje DMA kontrolera pre prenosa podataka

Pokretanje transfera podataka od akceleratora do DMA i od DMA do akceleratora se vrši pozivom funkcije **dmaStartTransfers()** čiji opis parametara (tabela 1.4) i deklaracija su dati u nastavku.

```
int dmaStartTransfers(u8* txBuff, u32 txSize, u8* rxBuff, u32 rxSize)
```

Čekanje da se transferi pokrenuti funkcijom **dmaStartTransfers()** se ostvaruje pozivom funkcije **dmaWaitTransfers()** čiji opis parametara (tabela 1.5) i deklaracija su dati u nastavku.

```
int dmaWaitTransfers(u32 txSize, u32 rxSize)
```

Tabela 1.4: Parametri funkcije `dmaStartTransfers()`

Parametri	Opis
<code>u8* txBuff</code>	Pokazivač na bafer u kome se nalaze podaci za prenos podataka od DMA do komponente akceleratora
<code>u32 txSize</code>	Broj bajtova za TX transfer
<code>u8* rxBuff</code>	Pokazivač na bafer u kome se nalaze podaci za prenos podataka od komponente akceleratora do DMA
<code>u32 rxSize</code>	Broj bajtova za RX transfer

Tabela 1.5: Parametri funkcije `dmaWaitTransfers()`

Parametri	Opis
<code>u32 txSize</code>	Očekivani broj bajtova u TX transferu
<code>u32 rxSize</code>	Očekivani broj bajtova u RX transferu

1.1.4 sd.h & sd.c

U ovom delu je realizovan "interfejs" koji omogućava korišćenje SD kartice. Potrebno je da se pozove funkcija `sdCardConfig()` koja inicijalizuje instancu SD kartice (`SdInstance`) pre njenog korišćenja. Pre gašenja programa potrebno je pozvati funkciju `sdCardUnmout()` koja "demonтира" karicu. U tabeli 1.6 je dat kratak opis funkcija.

Tabela 1.6: Funkcije za konfigurisanje SD kartice

Funkcija	Opis
<code>int sdCardConfig()</code>	Inicijalizacija i "montiranje" SD kartice
<code>void sdCardUnmout()</code>	"Demontiranje" SD kartice

Čitanje fajlova se vrši pozivom funkcije `sdReadFile()` čiji opis parametara (tabela 1.7) i deklaracija su dati u nastavku.

```
int sdReadFile(char* path, u8* buff, u32 size)
```

Tabela 1.7: Parametri funkcije `sdReadFile()`

Parametri	Opis
<code>char* path</code>	Putanja do fajla koji se čita
<code>u8* buff</code>	Pokazivač na bafer u kome se nalaze prostor za čitanje podataka
<code>u32 size</code>	Broj bajtova koje je potrebno pročitati

Upis u fajl se vrši pozivom funkcije `sdWriteFile()` čiji opis parametara (tabela 1.8) i deklaracija su dati u nastavku.

```
int sdWriteFile(char* path, u8* buff, u32 size)
```

Tabela 1.8: Parametri funkcije `sdWriteFile()`

Parametri	Opis
<code>char* path</code>	Putanja (uključujući i ime) na kojoj se kreira fajl za upis podataka
<code>u8* buff</code>	Pokazivač na bafer u kome se nalaze podaci za upis u fajl
<code>u32 size</code>	Broj bajtova koje je potrebno upisati

1.1.5 `processing.h` & `processing.c`

U ovim fajlovima je deklarirana i definisana funkcija `processing()` koja vrši softversko i pokreće hardversko procesiranje čiji opis parametara (tabela 1.9) i deklaracija su dati u nastavku.

```
int processing(u8* imBuff, u8* swBuff, u8* hwBuff, Params_t params)
```

Tabela 1.9: Parametri funkcije `processing()`

Parametri	Opis
<code>u8* imBuff</code>	Pokazivač na bafer gde su smešteni podaci za procesiranje
<code>u8* swBuff</code>	Pokazivač na bafer u koji se upisuju podaci SW procesiranja
<code>u8* hwBuff</code>	Pokazivač na bafer u koji se upisuju podaci HW procesiranja
<code>Params_t params</code>	Parametri procesiranja

U slučaju uspešnog procesiranja funkcija kao povratnu vrednost vraća `XST_SUCCESS`, u suprotnom vraća `XST_FAILURE`.

Unutar funkcije `processing()` prvo se pokreće tajmer (`startTimer()`) i započinje softversko procesiranje (`swProcessing()`). Nakon završenog procesiranja tajmer se zaustavlja (`stopTimer()`), ispisuje se izmereno vreme procesiranja (`getTime()`) i tajmer se resetuje (`resetTimer()`).

Pre hardverskog procesiranja prosleđuju trenutni parametri za procesiranje (`accConfig()`) preko AXI Lite interfejsa. Nakon čega se započinje procesiranje (`hwProcessing()`), DMA se resetuje (`dmaReset()`), pokreće se prenos podataka (`dmaStartTransfers()`) i čeka se na prijem obrađenih podataka (`dmaWaitTransfers()`). Merenje vremena procesiranja je identično kao kod softverskog procesiranja.

Nakon procesiranja se ispisuje razlika između vremena softverskog i hardverskog procesiranja, kao i ubrzanje hardverske obrade. Na kraju se proverava podudaranje podataka dva procesiranja (`checkData()`).

1.1.6 main.h

U **main.c** fajlu je realizovan glavni program koji se sastoji od inicijalizacije parametara, inicijalizacije hardvera (SD kartica, DMA i Timer) i glavne petlje koja služi za interakciju sa korisnikom.

Podrazumevani parametri su: **Mode** = **EDGE** (izračunavanje ivice slike), **EdgeThr** = **30**, **Border** = **CLOSEST_ELEM** (elementi van okvira slike imaju vrednost najbližeg elementa koji se nalazi u slici) i **Bypass** = **NO** (ne "propuštanje" slike).

Inicijalizacija hardvera se obavlja pozivom funkcije **hwConfid()**. Funkcija poziva funkcije za inicijalizaciju SD kartice (**sdCardConfig()**), tajmera (**timerConfig()**) i DMA komponente (**dmaConfig()**). U slučaju greške u ispisuje se poruka o neuspehom konfigurisanju određene komponente i vraća se **XST_FAILURE**, u suprotnom vraća se **XST_SUCCESS**.

Korisnik unosom broja od 0-8 bira neku od sledećih opcija:

1. Add image
2. Process image
3. Select mode
4. Change edge threshold
5. Change border parameter
6. Bypass processing
7. Print current parameters
8. Delete image (free memory)
0. Exit program

Prilikom dodavanja slike korisnik unosi naziv fajla (**imgName**), širinu (**ImgW**) i visinu (**ImgH**) slike u opsegu od 4 do **MAX_IMG_DIMENSION**, nakon čega se alokira prostor potreban za smeštanje vrednosti piksela slike. Slike (**.bin** fajlovi) se učitavaju sa SD kartice iz **images** foldera i smeštaju u prethodno alociranu memoriju.

Nakon učitavanja slike odabirom određenih opcija podešavaju se parametri za procesiranje. Odabirom opcije "2. Process image" pokreće se procesiranje, poziva se funkcija **processing()**. Nakon uspešnog procesiranja poziva se funkcija **saveProcessedImages()** koja čuva dobijene podatke i softverskog i hardverskog procesiranja u dva različita (**.bin**) fajla u folderu **processed** na SD kartici. Konvencija imenovanja fajlova je data u tabeli 1.10.

Tabela 1.10: Konvencija imenovanja izlaznih slika

Parametar	Vrednost	Dodatak imenu fajla
Bypass	YES NO	<code>_bypass</code> Na osnovu preostalih parametara
Mode	GRAD_H GRAD_V GRAD_M EDGE	<code>_gradH</code> <code>_gradV</code> <code>_gradM</code> <code>_edgeX</code> ($X = EdgeThr$)
Border	ZERO CLOSEST_ELEM	<code>_b0</code> <code>_b1</code>
Finalni naziv	SW HW	<code><ime_fajla>_sw.bin</code> <code><ime_fajla>_hw.bin</code>

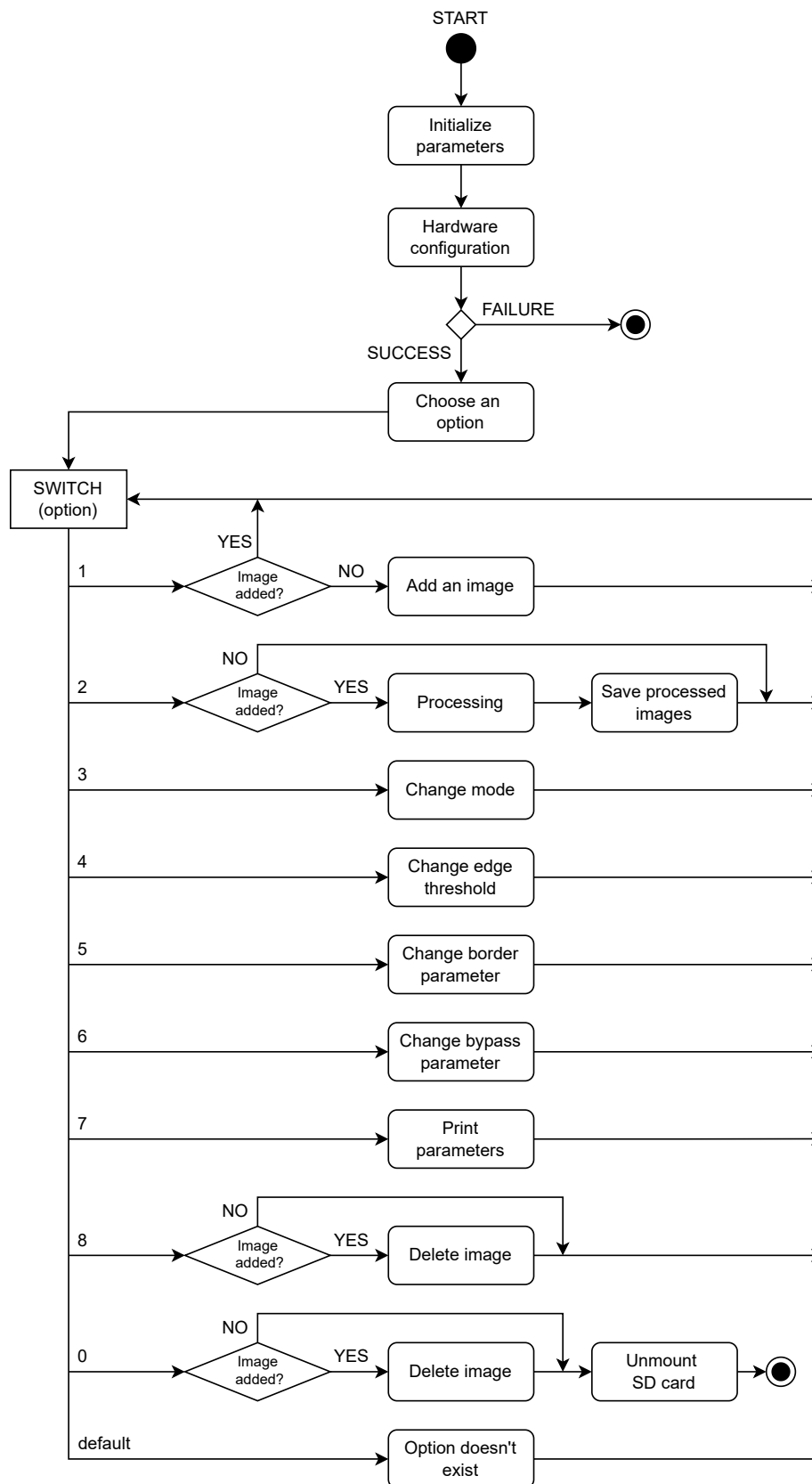
Primeri izlaznih fajlova:

- ◇ `img_name = lena, Bypass = YES`
`lena_bypass_sw.bin`
`lena_bypass_hw.bin`
- ◇ `img_name = lena, Mode = GRAD_H, Border = CLOSEST_ELEM`
`lena_gradH_b1_sw.bin,`
`lena_gradH_b1_hw.bin`
- ◇ `img_name = lena, Mode = EDGE, EdgeThr = 10, Border = ZERO`
`lena_edge10_b0_sw.bin,`
`lena_edge10_b0_hw.bin`

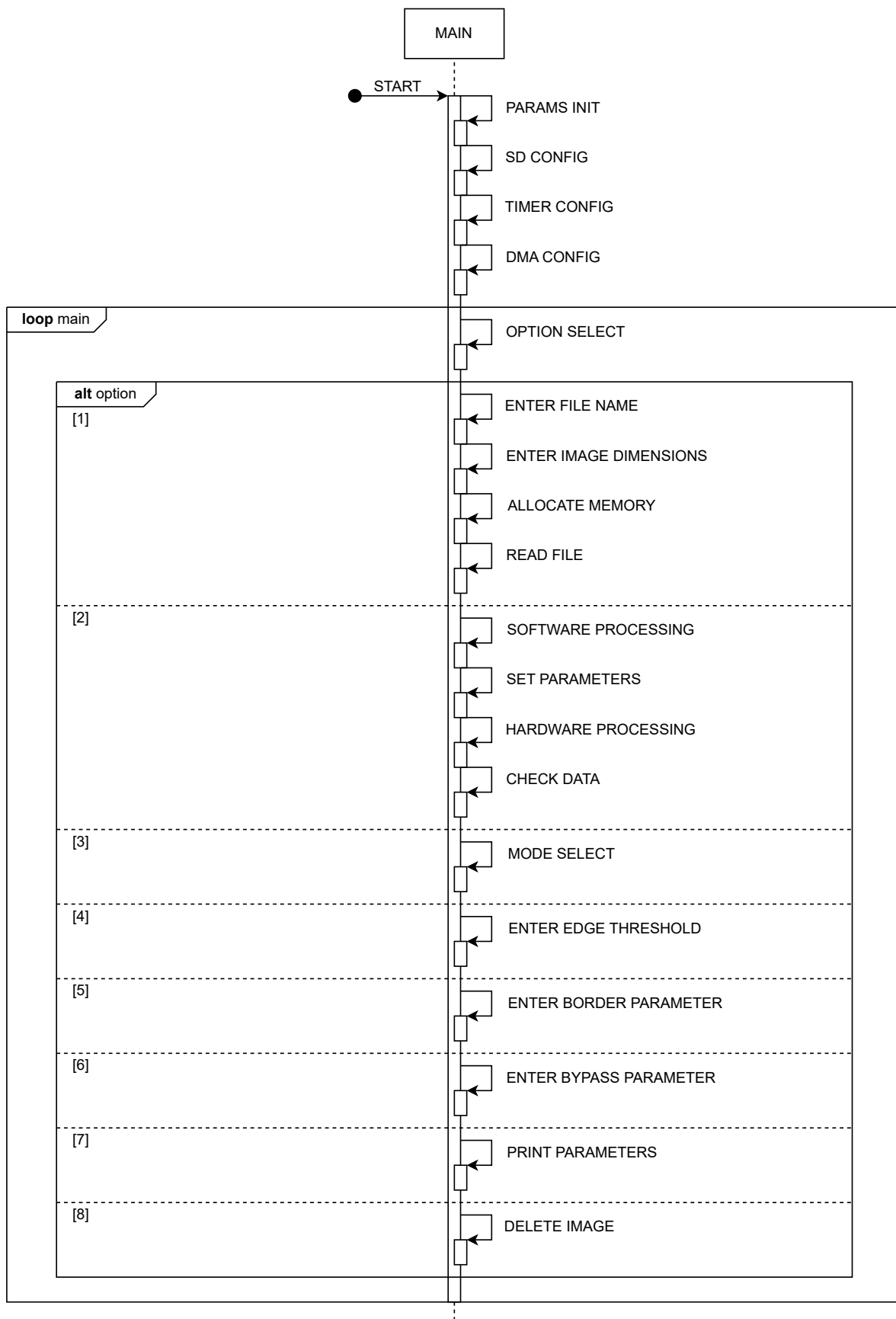
Ukoliko se dodaje nova slika, prethodno je potrebno da se stara slika obriše odabirom opcije "8. Delete image", čime se dealocira memorija.

1.2 Dijagrami

1.2.1 Dijagram aktivnosti



1.2.2 Dijagram sekvence



2 Hardver

2.1 Komponente

U ovoj sekciji su opisane realizovane komponente sistema i prikazani njihovi šematski simboli.

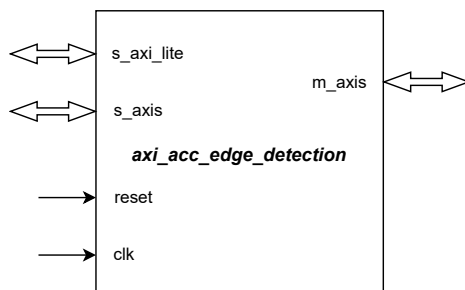
2.1.1 axi_acc_edge_detection

Komponenta u kojoj je realizovan *AXI Lite* interfejs i koja omogućava sinhronizaciju signala *AXI Stream* interfejsa (*slave* i *master*). Opis ulaznih i izlaznih signala je dat u tabeli 2.1.

Tabela 2.1: Interfejs *axi_acc_edge_detection* komponente

Signal	Broj bita	Smer	Opis
s_axi_lite	-	in/out	<i>AXI Lite slave</i> interfejs
s_axis	-	in/out	<i>AXI Stream slave</i> interfejs
m_axis	-	in/out	<i>AXI Stream master</i> interfejs

Šematski simbol je prikazan na slici 2.1.



Slika 2.1: Šematski simbol *axi_acc_edge_detection* komponente

2.1.2 acc_edge_detection

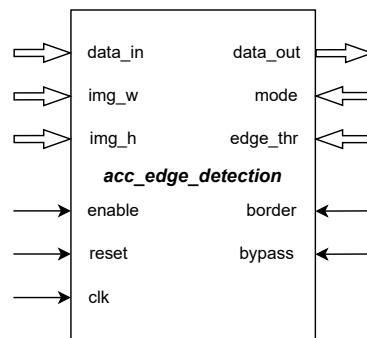
Komponenta u kojoj je realizovano procesiranje podataka. Opis ulaznih i izlaznih signala je dat u tabeli 2.2.

Tabela 2.2: Interfejs *acc_edge_detection* komponente

Signal	Broj bita	Smer	Opis
data_in	DATA_WIDTH	in	Ulazni podatak
enable	1	in	Dozvola/zaustavljanje obrade podataka
img_w	REG_WIDTH	in	Širina slike
img_h	REG_WIDTH	in	Visina slike
mode	2	in	Mode parametar

Signal	Broj bita	Smer	Opis
edge_thr	8	in	EdgThr parametar
border	1	in	Border parametar
bypass	1	in	Bypass parametar
data_out	DATA_WIDTH	out	Izlazni podatak

Šematski simbol je prikazan na slici 2.2.



Slika 2.2: Šematski simbol `acc_edge_detection` komponente

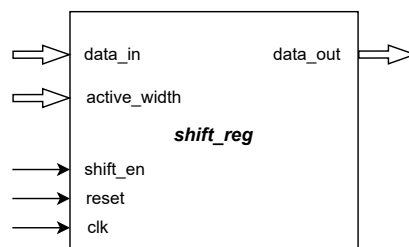
2.1.3 shift_reg

Shift registar realizovan tako da podatke upisuje i čita iz RAM memorije (BRAM). Opis ulaznih i izlaznih signala je dat u tabeli 2.3.

Tabela 2.3: Interfejs `shift_reg` komponente

Signal	Broj bita	Smer	Opis
data_in	DATA_WIDTH	in	Ulazni podatak
active_width	REG_WIDTH	in	Aktivna (korišćena) širina registra
shift_en	1	in	Dozvola šiftovanja
data_out	DATA_WIDTH	out	Izlazni podatak

Šematski simbol je prikazan na slici 2.3.



Slika 2.3: Šematski simbol `shift_reg` komponente

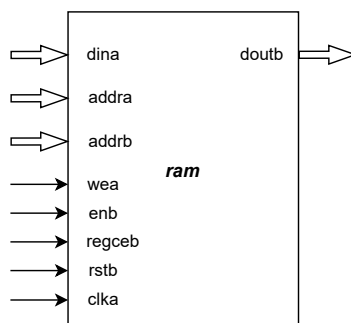
2.1.4 ram

Komponenta **ram** je realizovana tako da se implementira kao *Block RAM*. Opis ulaznih i izlaznih signala je dat u tabeli 2.4.

Tabela 2.4: Interfejs **ram** komponente

Signal	Broj bita	Smer	Opis
addra	$\log_2(\text{MAX_BUFF_LENGTH})$	in	Adresa upisa
addrb	$\log_2(\text{MAX_BUFF_LENGTH})$	in	Adresa čitanja
dina	DATA_WIDTH	in	Ulazni podatak
clka	1	in	Clock
wea	1	in	Dozvola upisa
enb	1	in	Dozvola za rad RAM-a (ušteda energije)
rstb	1	in	Resetovanje izlaza
regceb	1	in	Dozvola za izlazni registar
doutb	DATA_WIDTH	out	Izlazni podatak

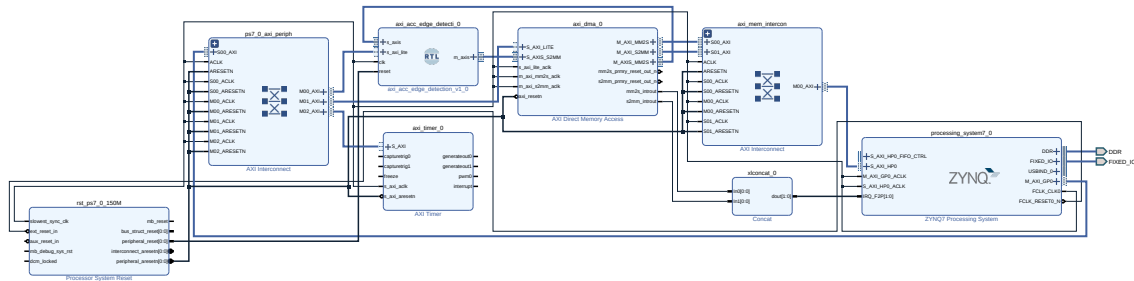
Šematski simbol je prikazan na slici 2.4.



Slika 2.4: **ram**

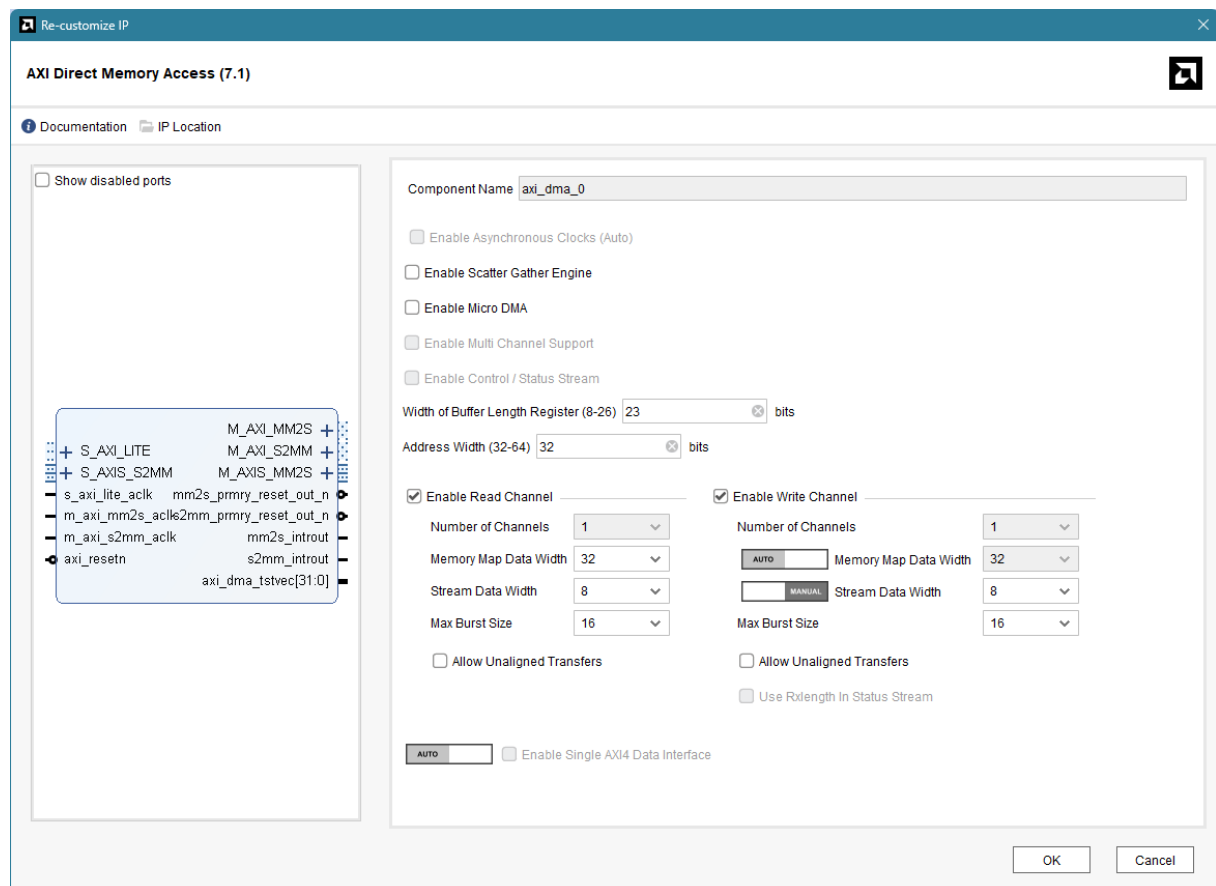
2.2 Šematski prikaz

U ovoj sekciji je prikazan šematski prikaz realizovanih komponenti sistema za detekciju ivica slike. Na slici 2.5 je prikazan dizajn celog sistema.



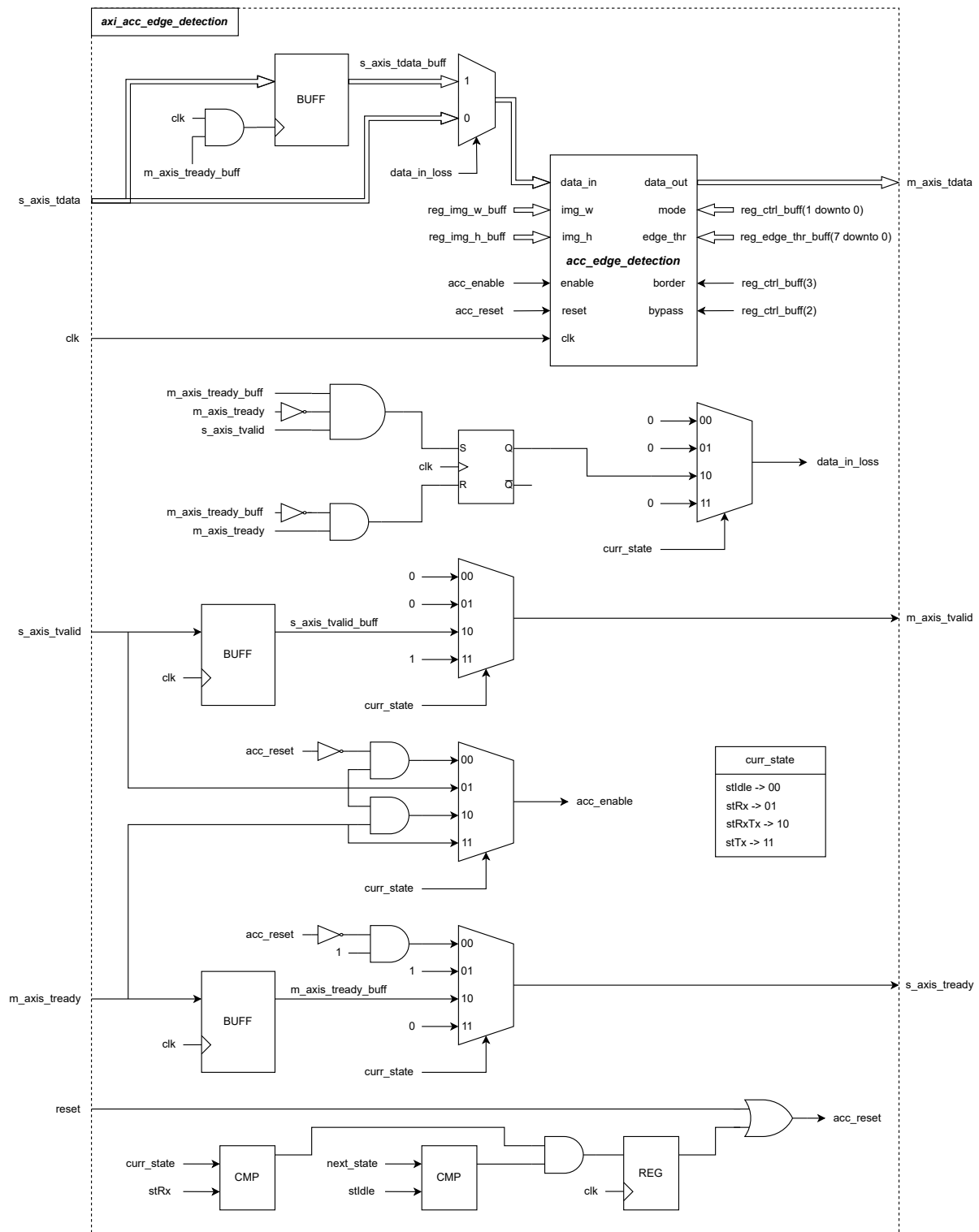
Slika 2.5: Blok dijagram sistema

Konfiguracija AXI DMA kontrolera je data na slici 2.6



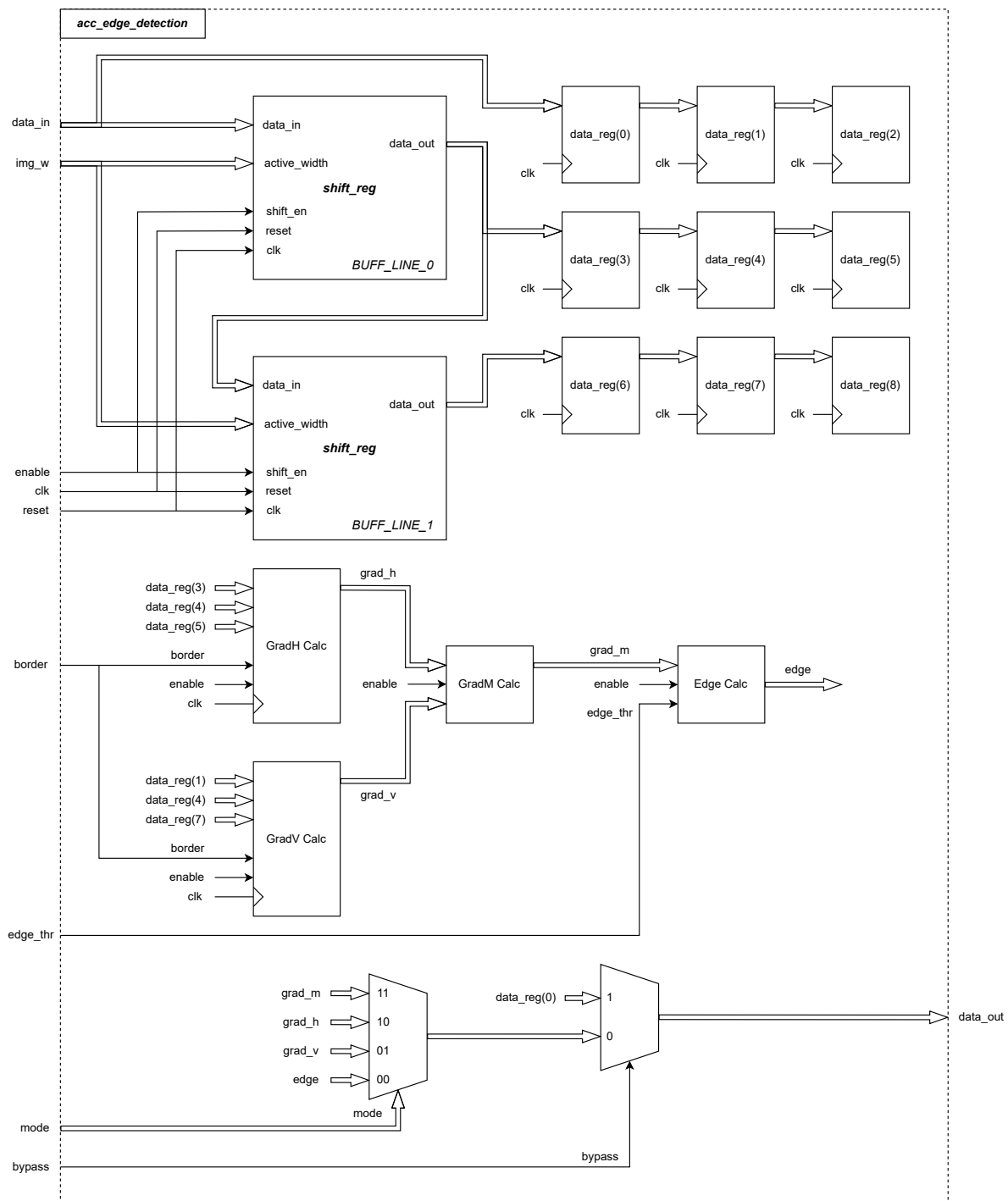
Slika 2.6: Konfiguracija AXI DMA

2.2.1 axi_acc_edge_detection



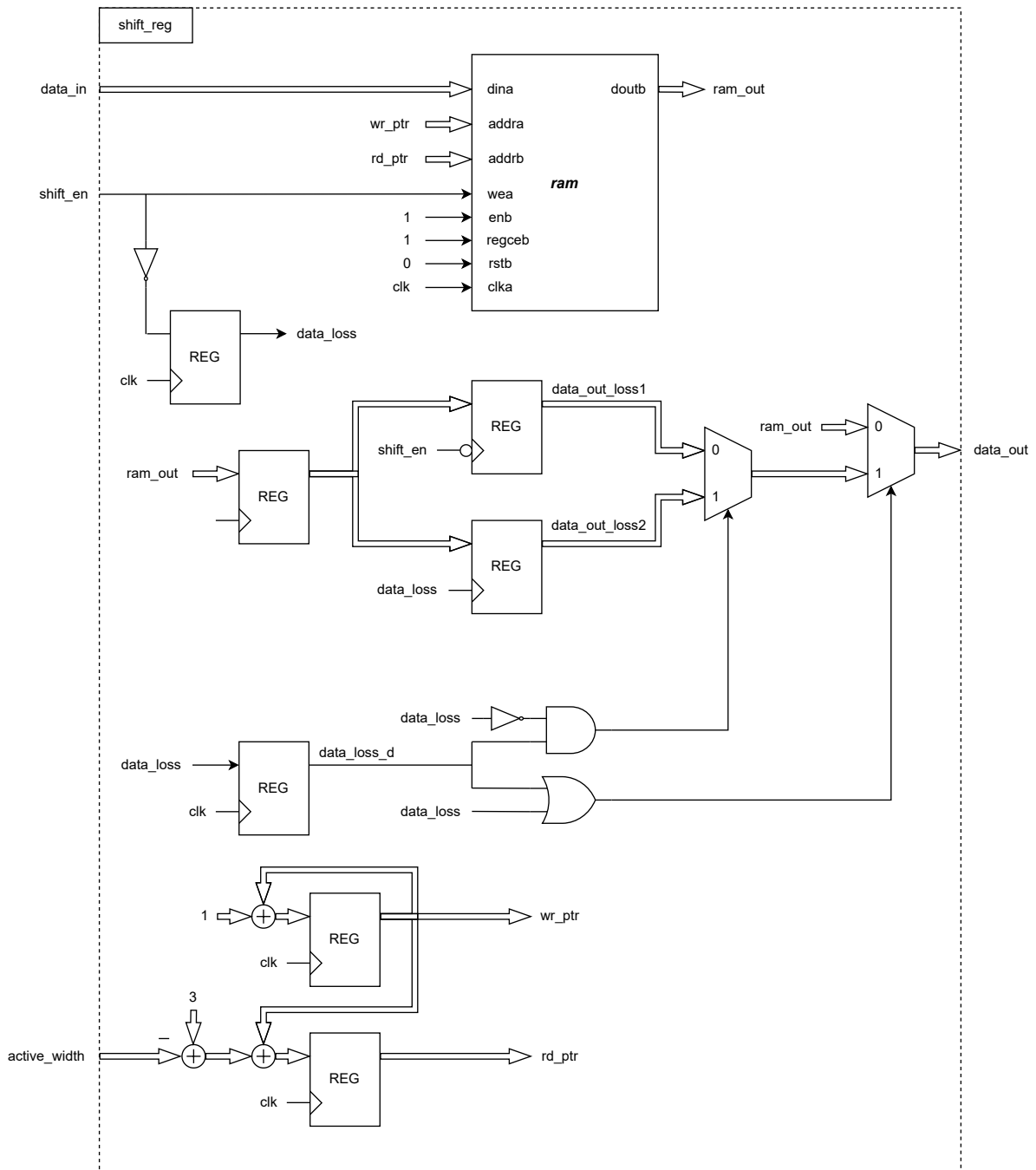
Slika 2.7: Šema axi_acc_edge_detection komponente

2.2.2 acc_edge_detection



Slika 2.8: Šema acc_edge_detection komponente

2.2.3 shift_reg

Slika 2.9: Šema `shift_reg` komponente

3 Testovi

3.1 Izlazne slike

Test 1 - *Bypassing*



Originalna slika



Softverska verzija



Hardverska verzija

Test 2 - Horizontalni gradijent



Originalna slika



Softverska verzija



Hardverska verzija

Test 3 - Vertikalni gradijent



Originalna slika



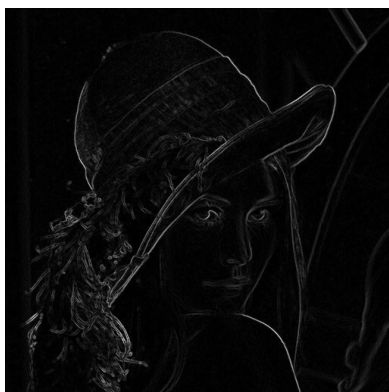
Softverska verzija



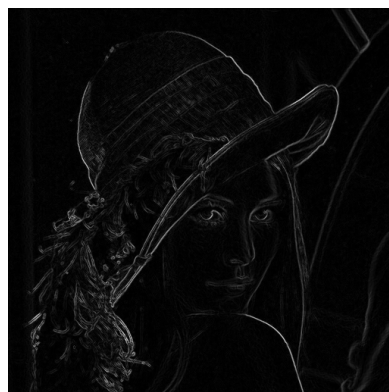
Hardverska verzija

Test 4 - Gradijent magnitude

Originalna slika



Softverska verzija



Hardverska verzija

Test 5 - Ivica slike

Originalna slika



Softverska verzija



Hardverska verzija

Test 6 - Slike različitih dimenzija

Originalna slika



Softverska verzija



Hardverska verzija



Originalna slika



Softverska verzija



Hardverska verzija



Originalna slika



Softverska verzija

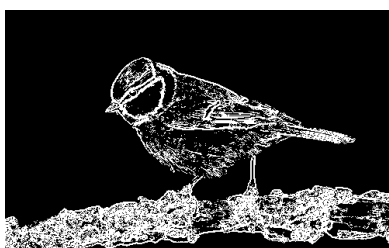


Hardverska verzija

Test 7 - Različite vrednosti praga ivice



Originalna slika



20



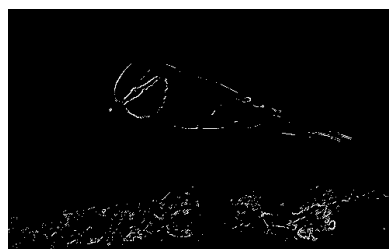
35



50



65



80

Test 8 - *Border*

Originalna slika



Najbliži element



Nula

3.2 Hardversko ubrzanje procesiranja

U tabeli 3.1 su data vremena softverskog i hardverskog procesiranja u mikrosekundama, kao i ubrzanje hardverskog u odnosu na softversko procesiranje. Takt FCLK_CLK0 na koji je povezana `axi_acc_edge_detection` komponenta iznosi 150 MHz.

Tabela 3.1: Rezultati procesiranja

Režim	Broj bajtova	Softver [us]	Hardver [us]	Razlika [us]	Ubrzanje
Bypass	99.600	2824	1784	1040	1.58x
	156.000	4418	1844	2574	2.40x
	195.072	5530	1885	3645	2.93x
	262.144	7410	1956	5454	3.79x
gradH	99.600	18383	1784	16599	10.30x
	156.000	28781	1844	26937	15.61x
	195.072	35999	1885	34114	19.10x
	262.144	48380	1956	46424	24.74x
gradV	99.600	18486	1784	16702	10.36x
	156.000	28970	1844	27126	15.71x
	195.072	36229	1885	34344	19.22x
	262.144	48696	1956	46740	24.90x
gradM	99.600	36467	1784	34683	20.44x
	156.000	57129	1844	55286	30.99x
	195.072	71451	1885	69566	37.90x
	262.144	96025	1956	94069	49.10x
edge	99.600	38640	1784	36856	21.66x
	156.000	60658	1844	58814	32.90x
	195.072	76078	1885	74193	40.36x
	262.144	101984	1956	100028	52.14x

